

EDUGENIE

SmartBridge 8-Phase Project Documentation

Google Gemini Powered Learning Assistant

**An AI-powered educational web application for Q&A, concept explanation,
quiz generation, summarization, and personalized learning paths.**

PROJECT OVERVIEW

EduGenie is a web-based educational assistant built with FastAPI and Google Gemini. It provides a single browser interface for asking questions, explaining concepts, summarizing educational text, generating multiple-choice quizzes, and producing practical learning-path recommendations.

The application separates the web/API layer from AI integration and feature modules. User requests are validated with Pydantic models and routed through FastAPI endpoints. A Gemini client handles real AI responses when configured, while the project also provides a `MOCK_AI` mode for local testing without an API key.

Main Technologies

- Python
- FastAPI
- Uvicorn
- HTML, CSS and JavaScript
- Jinja2 templates
- Google Gemini API / google-genai
- Pydantic
- python-dotenv
- HTTPX
- Pytest

Documented User Inputs

- Question for Q&A
- Topic for explanation
- Educational text for quiz generation
- Educational text for summarization
- Topic and learner level for learning-path recommendations
- Quiz question count: 3, 5 or 10 in the web interface; API validation permits 1–10

Documented Outputs

- Direct educational answer
- Beginner-friendly topic explanation
- Multiple-choice quiz with four options per question, correct answers and explanations
- Educational summary
- Structured learning-path recommendations

TEAM MEMBERS

Field	Information
Team ID	SWTID-2026-9696
Department / Course	BSc Computer Science
Batch	1

Name	NM ID	Email
Vaishnavi	BDD004D72468B92889B4D584EF8C5F20	vaishnavisuresh2907@gmail.com
Narmadha	54ACFFB275471868D1A4D6D211E0454F	narmadhas642@gmail.com
Jayasree	B6277900CD7E4EE903F4B8AA2D62CE7F	jayasreejaya816@gmail.com
Dhanalaksmi	3DF03A0574A1D69985F43E2BC0360ACA	abinayaabinaya1723@gmail.com
Anushya	D325FD5F76581F3B2404AF20F393B391	anushyaraj271@gmail.com

PHASE 1 – BRAINSTORMING & IDEATION PHASE

1. Background

EduGenie is designed around the need for a simple educational companion that can support common study activities in one interface. Instead of requiring separate tools for questions, explanations, summaries, quizzes and learning plans, the project combines these workflows behind a single FastAPI application.

2. Problem

- Students may need explanations in simpler language when learning technical or academic topics.
- Educational material can be lengthy and may need concise summaries for revision.
- Students can benefit from practice questions generated from study material.
- Learners may need a structured progression from beginner to advanced topics.
- Using separate tools for these activities can interrupt the learning workflow.

3. Proposed Solution

EduGenie provides a browser-based interface with five learning tasks: Q&A, Explain, Quiz, Summarize and Learning Path. Each task is mapped to a FastAPI endpoint and a dedicated Python module. Google Gemini supplies AI-generated responses when `MOCK_AI` is disabled and a valid API key is configured.

4. Objectives

- Provide direct educational answers through Q&A.
- Explain topics for beginners using definitions, examples, important points and a recap.
- Generate multiple-choice quizzes from supplied educational content.
- Summarize educational text for quick revision.
- Create practical learning paths based on topic and learner level.
- Provide a simple browser interface for all core learning functions.
- Support local development and testing through `MOCK_AI` mode.

5. Usage

1. Open the EduGenie homepage.
2. Select Q&A, Explain, Quiz, Summarize or Learning Path.
3. Enter the required content and, when applicable, select quiz count or learner level.
4. Submit the request.
5. The FastAPI backend validates the request and invokes the relevant feature module.
6. The result is displayed in the browser; quiz answers can be checked interactively.

6. Expected Outcome

The expected outcome is a working educational assistant that combines several AI-supported study functions into one web application, with a local mock mode for development and automated API tests for core routes.

PHASE 2 – REQUIREMENT ANALYSIS PHASE

1. Software Requirements

Software / Technology	Purpose / Documented Use
Python	Primary programming language and runtime.
FastAPI	Web framework and API routing.
Uvicorn	ASGI server for local execution.
Jinja2	Server-rendered HTML template support.
HTML / CSS / JavaScript	Browser interface and client-side interactions.
Google GenAI	Gemini client for AI generation.
Pydantic	Request and response validation.
python-dotenv	Loads environment configuration.
HTTPX	HTTP client dependency.
Pytest	Automated API testing.

2. Hardware / Environment Requirements

The project source does not specify exact CPU, RAM, storage or GPU requirements. The documented execution model is a Python environment with the required packages and a web browser. Real Gemini usage additionally requires network access and a valid Gemini API key.

3. Functional Requirements

- FR-01: Accept a question and return an educational answer.
- FR-02: Accept a topic and generate a beginner-friendly explanation.
- FR-03: Accept educational text and generate a quiz.
- FR-04: Validate quiz question count and quiz structure.
- FR-05: Accept educational text and generate a summary.
- FR-06: Accept a topic and learner level and generate learning recommendations.
- FR-07: Display task controls and generated results in the browser.
- FR-08: Provide copy functionality for generated results.
- FR-09: Expose a health endpoint for application status.
- FR-10: Support MOCK_AI mode for testing without a Gemini API key.

4. Non-Functional Requirements

- Usability: one interface provides five educational tasks.
- Maintainability: feature logic is separated into individual modules.
- Validation: Pydantic models enforce length and numeric constraints.
- Reliability: API routes use exception handling and return HTTP errors on failures.
- Testability: pytest tests are included for the main routes.
- Configuration: AI model, API key and mock mode are environment-driven.

5. User Requirements

- Ability to enter educational questions or text.
- Ability to select a learning task.
- Ability to select quiz question count.
- Ability to select beginner, intermediate or advanced level for learning paths.
- Ability to read and copy generated results.
- Ability to use the application locally.

PHASE 3 – PROJECT DESIGN PHASE

1. System Architecture

EduGenie follows a three-layer logical architecture: a browser frontend, a FastAPI application layer, and a Gemini integration layer. The frontend sends requests to FastAPI. FastAPI validates the request and delegates to a feature module. The feature module uses GeminiClient, which either calls Google Gemini or returns deterministic mock content when MOCK_AI is enabled.

Figure: System Architecture (textual representation)

Layer	Components	Responsibility
Frontend	index.html, style.css, app.js	Collect task input, submit requests, display results, handle quiz interaction and copy action.
API / Application	main.py, schemas.py, dependencies.py	Routing, validation, dependency injection, error handling and application configuration.
Feature Modules	qna.py, explanation_module.py, quiz_module.py, summary_module.py, learning_path.py	Build task-specific prompts and process AI output.
AI Integration	gemini_client.py, config.py	Configure Gemini model/API access, generate text/JSON and provide MOCK_AI behavior.

2. Components Definition

Component	Purpose	Interaction
Frontend	Provides five task modes and result display.	Calls /qa, /explain, /quiz, /summarize and /learn/recommendations.
FastAPI Backend	Receives and validates API requests.	Delegates to feature modules and returns JSON.
GeminiClient	Central AI service abstraction.	Uses Google GenAI when MOCK_AI=false; otherwise uses mock responses.
Q&A Module	Answers questions clearly.	Calls client.generate().
Explanation Module	Explains topics to beginners.	Calls client.generate().
Quiz Module	Creates and validates MCQs.	Calls client.generate_json() and validates QuizResponse.
Summary Module	Creates revision summaries.	Calls client.generate().
Learning Path Module	Creates progression plans.	Calls client.generate().

3. Data Flow

- User selects a task and enters content in the browser.
- JavaScript builds the appropriate JSON request.
- FastAPI receives the request.
- Pydantic validates the request schema.
- The selected module creates an AI prompt.
- GeminiClient calls Gemini or MOCK_AI.
- The route returns a JSON response.
- JavaScript renders the result in the output card.
- For quizzes, the browser lets the user select answers and calculates a local score.

4. User Flow

16. Open homepage
17. Choose a task
18. Enter content
19. Optionally choose quiz count or learning level
20. Click Generate
21. Wait for the response
22. Read/copy the result
23. For quizzes, select answers and click Check Answers

5. Database Design

No database, tables, collections, ORM, or persistent database connection is present in the supplied project. EduGenie processes requests and returns generated results without a documented persistent user or learning-history database.

6. Module Design

Module / File	Documented Function
main.py	Creates the FastAPI app, mounts static files, renders homepage and defines API routes.
schemas.py	Defines Pydantic request and quiz response models.
config.py	Loads application name, Gemini model, API key and MOCK_AI setting.
gemini_client.py	Provides generate(), generate_json() and mock-generation behavior.
qna.py	Defines answer_question().
explanation_module.py	Defines explain_topic().
quiz_module.py	Defines generate_quiz() and validates generated quiz structure.
summary_module.py	Defines summarize_text().
learning_path.py	Defines get_learning_recommendations().
dependencies.py	Provides the GeminiClient dependency.
templates/index.html	Main browser interface.
static/app.js	Task switching, API requests, rendering, quiz scoring and copy behavior.
static/style.css	Frontend styling.

7. Security Design

- Gemini API credentials are intended to be supplied through environment configuration.
- The supplied project contains .env.example and an environment-driven configuration design.
- Pydantic request models enforce input length and numeric constraints.
- FastAPI routes catch exceptions and return HTTP 500 errors.
- The project does not document authentication, user accounts, password storage, authorization or persistent session management.

PHASE 4 – PROJECT PLANNING PHASE

1. Project Modules

- Frontend interface
- FastAPI API layer
- Request/response schemas
- Gemini integration
- Q&A
- Topic explanation
- Quiz generation and validation
- Text summarization
- Learning-path recommendations
- Automated testing
- Local deployment

2. Entity / Data Structure

The main structured data types are request and response models defined with Pydantic.

Model	Fields
QuestionRequest	question
ExplainRequest	topic
TextRequest	text
QuizRequest	text, number_of_questions
LearningPathRequest	topic, level
QuizQuestion	question, options, correct_answer, explanation
QuizResponse	title, questions

3. Key Features

Feature	Description
Q&A	Answers student questions with direct explanations and examples when useful.
Explain	Uses a defined beginner explanation structure.
Quiz	Generates MCQs and validates four options and a matching correct answer.
Summarize	Produces concise educational summaries for revision.
Learning Path	Generates prerequisites, progression, practice, projects, milestones and revision strategy.
Mock AI	Allows local testing without a Gemini API key.
Quiz Scoring	Calculates the user's selected-answer score in the browser.
Copy Result	Copies generated result text to the clipboard.

4. User Roles and Responsibilities

Role	Documented Responsibility
End User / Student	Selects a learning function, enters content, views results and uses quiz scoring/copy functionality.
Developer / Tester	Runs the local application, configures environment variables and executes the automated tests.
Admin	No authenticated admin role is documented.

5. Team Responsibilities

The supplied team information contains names, NM IDs and email addresses, but does not provide role assignments. Therefore, responsibilities are intentionally left unassigned rather than inferred.

6. Development Timeline

The supplied source does not contain calendar dates or a formal milestone schedule. A source-grounded development sequence is:

24. Set up Python environment and dependencies
25. Create FastAPI application and schemas
26. Implement Gemini client and MOCK_AI mode
27. Implement individual educational modules
28. Build browser interface
29. Connect frontend to API routes
30. Add automated tests
31. Prepare local execution and documentation

PHASE 5 – PROJECT DEVELOPMENT PHASE

1. Development Environment Setup

The supplied README documents Windows execution using a virtual environment, dependency installation, pytest execution and Uvicorn.

```
python -m pip install -r requirements.txt
```

```
python -m pytest -q
```

```
python -m uvicorn main:app --reload
```

Then open: `http://127.0.0.1:8000`

The README states that the included environment starts in MOCK_AI mode, so an API key is not required for testing. To use Gemini, GEMINI_API_KEY must be set and MOCK_AI=false.

2. Project Folder Structure

```
EduGenie/
|— main.py
|— config.py
|— dependencies.py
|— gemini_client.py
|— schemas.py
|— qna.py
|— explanation_module.py
|— quiz_module.py
|— summary_module.py
|— learning_path.py
|— requirements.txt
|— README.md
|— templates/
|   |— index.html
|— static/
|   |— style.css
|   |— app.js
|— tests/
|   |— test_api.py
|— .vscode/
```

3. Installation and Configuration

- Create/activate a Python virtual environment.
- Install dependencies from requirements.txt.
- Use the provided environment configuration pattern.
- For real Gemini responses, configure GEMINI_API_KEY and set MOCK_AI=false.
- Run the FastAPI app with Uvicorn.

4. Database Implementation

No database implementation is documented in the supplied project. EduGenie does not persist user accounts, quiz history, summaries or learning paths in a database.

5. Module Implementation

Module 1: Q&A

`answer_question()` sends the student's question to GeminiClient with instructions to answer accurately, give a direct answer first, explain simply and avoid invented facts.

Module 2: Explanation

`explain_topic()` requests a beginner-oriented explanation with a simple definition, how it works, an easy example, important points and a short recap.

Module 3: Quiz

`generate_quiz()` requests exactly the selected number of MCQs, validates the QuizResponse, checks that every question has four options and verifies that the correct answer matches an option.

Module 4: Summary

`summarize_text()` asks the AI to retain important information and technical terms while removing repetition and using headings or bullets where useful.

Module 5: Learning Path

`get_learning_recommendations()` requests prerequisites, beginner/intermediate/advanced topics, timeline, practice activities, project ideas, resource types, milestones and revision strategy.

Module 6: FastAPI Routing

`main.py` defines the homepage and the `/health`, `/qa`, `/explain`, `/quiz`, `/summarize` and `/learn/recommendations` endpoints.

6. Core Functionality

Route	Purpose
GET /	Renders the EduGenie homepage.
GET /health	Returns application status, mock mode and model information.
POST /qa	Answers a student question.
POST /explain	Explains a topic.
POST /quiz	Generates a validated quiz.
POST /summarize	Summarizes educational text.
POST /learn/recommendations	Generates learning recommendations.

7. Authentication / Security Implementation

No user authentication or account-management implementation is documented. Security-related controls in the source include environment-based API configuration, Pydantic validation and route-level exception handling.

8. User Interface Implementation

The single `index.html` page provides navigation buttons for Q&A, Explain, Quiz, Summarize and Learning Path. The form dynamically changes its label and placeholder. Quiz mode displays a question-count selector, while Learning Path mode displays a level selector.

Screenshot placeholder: EduGenie homepage / task selection and input form.

Screenshot placeholder: EduGenie generated result view.

Screenshot placeholder: EduGenie quiz and score view.

9. Integration

Integration	Documented Implementation
Frontend ↔ Backend	app.js sends JSON requests to FastAPI endpoints and renders responses.
Backend ↔ Feature Modules	main.py delegates to the corresponding Python feature function.
Feature Modules ↔ Gemini	Modules use GeminiClient.generate() or generate_json().
Gemini ↔ Google API	When MOCK_AI=false, google-genai is used with the configured Gemini model.
Frontend ↔ Quiz Logic	Quiz answers are checked in browser JavaScript using the returned correct_answer values.

10. Project Execution

The documented local execution command is:

```
python -m uvicorn main:app --reload
```

Local application: http://127.0.0.1:8000

PHASE 6 – PROJECT TESTING PHASE

1. Testing Objectives

- Verify homepage loading
- Verify health endpoint
- Verify Q&A endpoint
- Verify explanation endpoint
- Verify summarization endpoint
- Verify learning-path endpoint
- Verify quiz generation and expected question count
- Verify request/response behavior under MOCK_AI mode

2. Testing Environment

Item	Documented Information
Runtime	Python
Web Framework	FastAPI
Server	Uvicorn
Testing Framework	Pytest
AI Integration	Google GenAI / Gemini; MOCK_AI for local tests
Database	No database documented
Local URL	http://127.0.0.1:8000

3. Testing Methods

- Functional API testing
- Route-level testing
- Basic UI/homepage response testing
- Validation testing for quiz output structure
- Health/status testing

4. Test Cases

Test ID	Test Scenario	Input	Expected Output	Source Status
TC-01	Load homepage	GET /	Homepage contains EduGenie	Defined in test_api.py
TC-02	Health check	GET /health	HTTP 200	Defined in test_api.py
TC-03	Q&A	POST /qa with SQL question	HTTP 200	Defined in test_api.py
TC-04	Explain	POST /explain with SQL topic	HTTP 200	Defined in test_api.py
TC-05	Summarize	POST /summarize with short text	HTTP 200	Defined in test_api.py
TC-06	Learning path	POST /learn/recommendations	HTTP 200	Defined in test_api.py
TC-07	Quiz	POST /quiz with 5 questions	HTTP 200 and 5 questions	Defined in test_api.py

5. Testing Results

The supplied test file contains seven automated test functions covering the main application routes. During preparation of this documentation, an execution attempt in the documentation environment could not collect the tests

because the google-genai package was not installed in that environment. Therefore, this document does not claim that the suite passed. The source code itself is the basis for the test-case descriptions above.

6. UI Testing

- Task navigation is implemented in app.js.
- Input labels and placeholders change by task.
- Quiz mode provides 3/5/10 question choices.
- Learning Path mode provides beginner/intermediate/advanced choices.
- Generated results are displayed in the result card.
- Copy action uses the browser clipboard API.
- Quiz scoring is calculated in the browser.

7. Integration Testing

The project contains explicit integration points between JavaScript and FastAPI, FastAPI and feature modules, and feature modules and GeminiClient. Formal measured integration-test results are not separately documented.

8. Error Handling Testing

API routes wrap feature execution in try/except blocks and raise HTTPException with status 500 when unexpected exceptions occur. Quiz generation also validates question count, option count and correct-answer membership.

9. Performance Testing

No formal response-time, throughput, load or resource-utilization benchmarks are included in the supplied project.

10. Security Testing

No formal security test report is included. The source does provide environment-based API configuration and Pydantic input validation.

11. Bug Fixes

No formal bug log or bug-fix register is included in the supplied project.

PHASE 7 – PROJECT DOCUMENTATION PHASE

1. Project Introduction

EduGenie is an AI-powered educational assistant that combines question answering, concept explanation, quiz generation, summarization and learning-path recommendations in a single FastAPI web application.

2. Problem Statement

Students use different study workflows: understanding unfamiliar concepts, revising long material, practicing questions and planning what to learn next. EduGenie addresses these related activities through a unified educational assistant.

3. Objectives

- Answer educational questions
- Explain concepts simply
- Generate practice quizzes
- Summarize educational content
- Recommend structured learning paths
- Provide a simple browser-based experience

4. Technologies Used

Python, FastAPI, Uvicorn, Jinja2, HTML, CSS, JavaScript, Google GenAI, Gemini, Pydantic, python-dotenv, HTTPX and Pytest.

5. System Requirements

A Python environment with the project dependencies, a web browser and network access for real Gemini usage are required. The project does not specify exact hardware requirements.

6. System Design

The system begins with a browser request, routes it through FastAPI and Pydantic validation, delegates to a specialized educational module, obtains an AI response through GeminiClient, and returns the result to the browser.

7. Implementation

The implementation is modular: routing is in `main.py`, schemas are in `schemas.py`, configuration is in `config.py`, AI access is in `gemini_client.py`, and each educational capability has its own Python module.

8. Features

- Q&A
- Explain
- Quiz generation
- Quiz validation
- Quiz scoring
- Summarization
- Learning paths
- Copy result
- Mock AI mode
- Health endpoint

9. User Roles

The current source documents end-user/student interaction and developer/tester execution. No authenticated role hierarchy is implemented.

10. Testing Summary

Automated tests are supplied for homepage, health, Q&A, explanation, summarization, learning recommendations and quiz generation. Formal pass/fail results are not asserted in this document because the available execution environment could not import google-genai.

11. Advantages

- Combines several study functions in one interface.
- Supports real Gemini responses and a mock mode.
- Uses modular feature files.
- Uses Pydantic validation for API requests and quiz responses.
- Includes automated API tests.
- Provides interactive quiz scoring in the browser.

12. Limitations

- No database or persistent learning history is documented.
- No authentication or user accounts are documented.
- No formal performance benchmarks are provided.
- No formal bug log is provided.
- Real AI responses require a valid Gemini API key and network access.
- The supplied source does not document production deployment.

13. Future Enhancements

- User accounts and authentication
- Saved quiz and learning history
- Progress dashboards
- Persistent learning plans
- More learning formats such as flashcards
- Teacher/admin features
- Database-backed personalization
- Production deployment and monitoring

14. Conclusion

EduGenie demonstrates a modular AI-assisted learning workflow. Its FastAPI backend exposes focused educational endpoints, the browser interface provides a simple task-oriented experience, and GeminiClient separates AI access from the educational modules. The project also includes a MOCK_AI mode and a pytest suite to support local development and testing.

PHASE 8 – PROJECT DEMONSTRATION PHASE

1. Project Introduction

Begin the demonstration by introducing EduGenie, the team, and the goal of combining common educational assistance functions into one web application.

2. Project Overview

Explain that the user can choose Q&A, Explain, Quiz, Summarize or Learning Path, provide the relevant content, generate a result and interact with the returned content.

3. Project File / Code Structure

Show main.py, config.py, schemas.py, gemini_client.py, the five feature modules, templates/index.html, static/app.js, static/style.css, requirements.txt, README.md and tests/test_api.py.

4. Project Execution

32. Activate the Python virtual environment.
33. Install dependencies.
34. Run pytest when dependencies are installed.
35. Start the application with `python -m uvicorn main:app --reload`.
36. Open `http://127.0.0.1:8000`.

5. Feature Demonstration

37. Demonstrate Q&A with an educational question.
38. Demonstrate Explain with a topic.
39. Paste study material into Quiz and select the number of questions.
40. Answer the generated MCQs and show the score.
41. Paste text into Summarize and show the result.
42. Enter a topic and level in Learning Path and show the generated progression.
43. Use Copy on a generated result.

6. Complete User Workflow

44. Homepage → select task
45. Input → enter question/topic/text
46. Frontend → sends JSON request
47. FastAPI → validates request
48. Feature module → prepares prompt
49. GeminiClient → Gemini or MOCK_AI
50. API → returns response
51. Frontend → displays result
52. Quiz mode → user answers and checks score

7. Final Output

The final demonstration should show the EduGenie homepage and at least one successful example of each major feature. For real AI demonstration, configure `GEMINI_API_KEY` and set `MOCK_AI=false`.

8. Benefits / Use of the Project

- Supports students during study and revision.

- Combines multiple educational workflows.
- Can generate practice questions from supplied content.
- Can provide structured learning progressions.
- Can operate locally in MOCK_AI mode for development.

9. Conclusion

The demonstration concludes by showing the end-to-end path from educational input to AI-assisted output across the five EduGenie functions.

10. Demo Video

Demo video link: https://drive.google.com/drive/folders/1oAzUVsKAtLOKiEQcpE8MfXay4dSxx_y9

APPENDIX A – API REFERENCE

Method	Endpoint	Request Body / Purpose
GET	/	Render homepage
GET	/health	Application health and configuration status
POST	/qa	{"question":"..."}
POST	/explain	{"topic":"..."}
POST	/quiz	{"text":"...", "number_of_questions":3}
POST	/summarize	{"text":"..."}
POST	/learn/recommendations	{"topic":"...", "level":"beginner"}

END OF DOCUMENTATION